



Compilers: Principles, Techniques, and Tools

Min ZHANG

mzhang@sei.ecnu.edu.cn

Lexical Analysis

From RE to DFA



Converting Regular Expressions Directly to DFAs

- ◆ We may convert a regular expression into a DFA (without creating a NFA first).
- ◆ First we augment the given regular expression by concatenating it with a special symbol #.

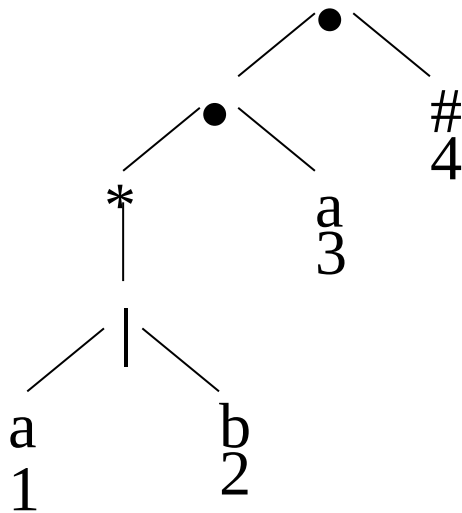
$r \rightarrow (r)\#$ augmented regular expression

- ◆ Then, we create a syntax tree for this augmented regular expression.
- ◆ In this syntax tree, all alphabet symbols (plus # and the empty string) in the augmented regular expression will be on the leaves, and all inner nodes will be the operators in that augmented regular expression.
- ◆ Then each alphabet symbol (plus #) will be numbered (position numbers).

Regular Expression $\rightarrow$ DFA



$(a|b)^* a \rightarrow (a|b)^* a \#$ augmented regular expression



Syntax tree of $(a|b)^* a \#$

- each symbol is numbered (positions)
- each symbol is at a leaf
- inner nodes are operators

followpos

Then we define the function **followpos** for the positions (positions assigned to leaves).



followpos(i) -- is the set of positions which can follow the position i in the strings generated by the augmented regular expression.

For example, $(a \mid b)^* a \#$
1 2 3 4

$\text{followpos}(1) = \{1, 2, 3\}$

$\text{followpos}(2) = \{1, 2, 3\}$

$\text{followpos}(3) = \{4\}$

$\text{followpos}(4) = \{\}$

*followpos is just defined for leaves,
it is not defined for inner nodes.*

firstpos, lastpos, nullable

- ◆ To evaluate followpos, we need three more functions to be defined for the nodes (not just for leaves) of the syntax tree.
- ◆ $\text{firstpos}(n)$ -- the set of the positions of the first symbols of strings generated by the sub-expression rooted by n .
- ◆ $\text{lastpos}(n)$ -- the set of the positions of the last symbols of strings generated by the sub-expression rooted by n .
- ◆ $\text{nullable}(n)$ -- *true* if the empty string is a member of strings generated by the sub-expression rooted by n .
false otherwise.

How to evaluate firstpos, lastpos, nullable

<u>n</u>	<u>nullable(n)</u>	<u>firstpos(n)</u>	<u>lastpos(n)</u>
leaf labeled ϵ	true	Φ	Φ
leaf labeled with position i	false	{i}	{i}
$ \begin{array}{c} \quad / \quad \backslash \\ c_1 \quad c_2 \end{array} $	nullable(c_1) or nullable(c_2)	firstpos(c_1) $\cup$ firstpos(c_2)	lastpos(c_1) $\cup$ lastpos(c_2)
$ \begin{array}{c} \bullet \quad / \quad \backslash \\ c_1 \quad c_2 \\ \\ \end{array} $	nullable(c_1) and nullable(c_2)	if (nullable(c_1)) firstpos(c_1) $\cup$ firstpos(c_2) else firstpos(c_1)	if (nullable(c_2)) lastpos(c_1) $\cup$ lastpos(c_2) else lastpos(c_2)
$ \begin{array}{c} * \\ c_1 \end{array} $	true	firstpos(c_1)	lastpos(c_1)

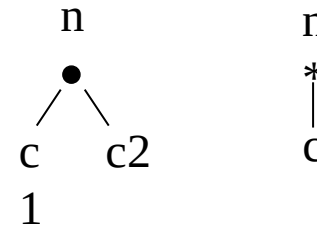
Fig.3.40. rules for computing nullable and firstpos.

How to evaluate followpos



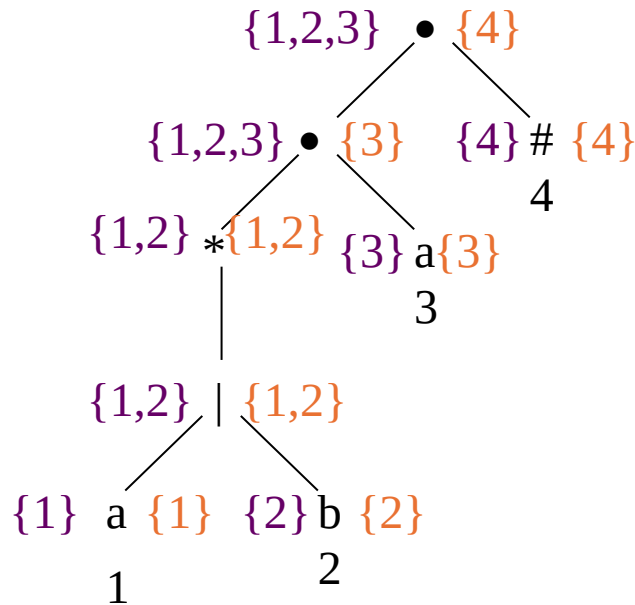
◆ Two-rules define the function followpos:

1. If n is concatenation-node with left child c_1 and right child c_2 , and i is a position in $\text{lastpos}(c_1)$, then all positions in $\text{firstpos}(c_2)$ are in $\text{followpos}(i)$.
2. If n is a star-node, and i is a position in $\text{lastpos}(n)$, then all positions in $\text{firstpos}(n)$ are in $\text{followpos}(i)$.



◆ If firstpos and lastpos have been computed for each node, followpos of each position can be computed by making one depth-first traversal of the syntax tree.

Example – $(a \mid b)^* a \#$



green – firstpos
blue – lastpos

Then we can calculate followpos

$\text{followpos}(1) = \{1,2,3\}$

$\text{followpos}(2) = \{1,2,3\}$

$\text{followpos}(3) = \{4\}$

$\text{followpos}(4) = \{\}$

- ◆ After we calculate follow positions, we are ready to create DFA for the regular expression.

Algorithm (RE $\rightarrow$ DFA)



- ◆ Create the syntax tree of $(r) \#$
- ◆ Calculate the functions: followpos, firstpos, lastpos, nullable
- ◆ Put firstpos(root) into the states of DFA as an unmarked state.

while (there is an unmarked state S in the states of DFA) do

- *mark S*
- *for each input symbol a do*
 - *let $s_1, \dots, s_n$ are positions in S and symbols in those positions are a*
 - *$S' \leftarrow \text{followpos}(s_1) \cup \dots \cup \text{followpos}(s_n)$*
 - *$\text{move}(S, a) \leftarrow S'$*
 - *if (S' is not empty and not in the states of DFA)*
 - *put S' into the states of DFA as an unmarked state.*

- *the start state of DFA is firstpos(root)*
- *the accepting states of DFA are all states containing the position of $\#$*

Example – (a ₁ | b ₂) * a ₃ # ₄

followpos(1)={1,2,3} followpos(2)={1,2,3} followpos(3)={4} followpos(4)={}

$S_0 = \text{firstpos}(\text{root}) = \{1, 2, 3\}$

⇓ mark S_0

a: followpos(1) $\cup$ followpos(3)={1,2,3,4}= S_1

move(S_0 ,a)= S_1

b: followpos(2)={1,2,3}= S_0

move(S_0 ,b)= S_0

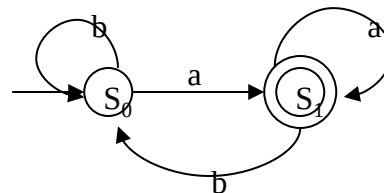
⇓ mark S_1

a: followpos(1) $\cup$ followpos(3)={1,2,3,4}= S_1

move(S_1 ,a)= S_1

b: followpos(2)={1,2,3}= S_0

move(S_1 ,b)= S_0



start state: S_0

accepting states: $\{S_1\}$

	a	b
S0	S1	S0
S1	S1	S0

Exercise – $(a \mid \varepsilon) b c^* \#$



Please compute firstpos, lastpos, followpos,
then convert to DFA

Exercise and answer -- (a | ϵ) b c^{*}



1 2 3 4

followpos(1)={2} followpos(2)={3,4}
followpos(3)={3,4} followpos(4)={}

$S_0 = \text{firstpos}(\text{root}) = \{1, 2\}$

$\Downarrow$ mark S_0

a: followpos(1)={2}= S_1

move(S_0 , a)= S_1

b: followpos(2)={3,4}= S_2

move(S_0 , b)= S_2

$\Downarrow$ mark S_1

b: followpos(2)={3,4}= S_2

move(S_1 , b)= S_2

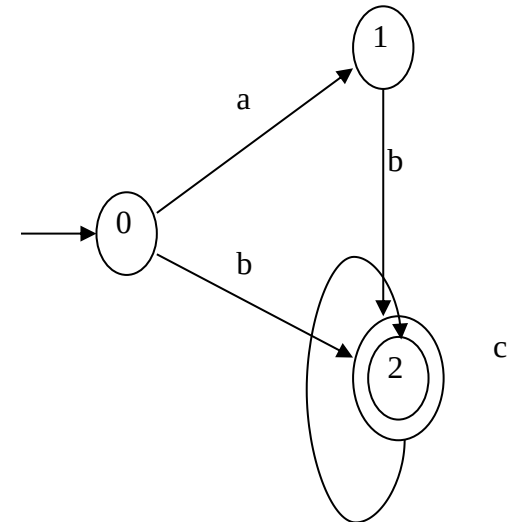
$\Downarrow$ mark S_2

c: followpos(3)={3,4}= S_2

move(S_2 , c)= S_2

start state: S_0

accepting states: $\{S_2\}$



Minimizing the Number of States of a DFA



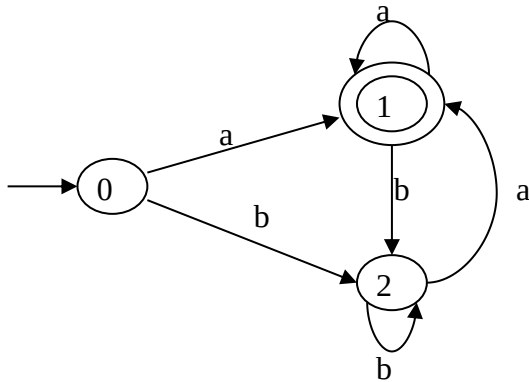
- ◆ partition the set of states into two groups:
 - ◆ G_1 : set of accepting states
 - ◆ G_2 : set of non-accepting states
- ◆ For each new group G
 - ◆ partition G into subgroups such that states s_1 and s_2 are in the same group if
 - ◆ for all input symbols a , states s_1 and s_2 have transitions to states in the same group.
- ◆ Start state of the minimized DFA is the group containing the start state of the original DFA.
- ◆ Accepting states of the minimized DFA are the groups containing the accepting states of the original DFA.

Minimizing the Number of States of DFA



1. Construct initial partition Π of S with two groups: accepting/ non-accepting.
2. (Construct Π_{new}) For each group G of Π **do begin**
 - ① Partition G into subgroups such that two states s, t of G are in the same subgroup if for all symbols a states s, t have transitions on a to states of the same group of Π .
 - ② Replace G in Π_{new} by the set of all these subgroups.
3. Compare Π_{new} and Π . If equal, $\Pi_{\text{final}} := \Pi$ then proceed to 4, else set $\Pi := \Pi_{\text{new}}$ and goto 2.
4. Aggregate states belonging in the groups of Π_{final}

Minimizing DFA - Example



$$G_1 = \{1\}$$

$$G_2 = \{0, 2\}$$

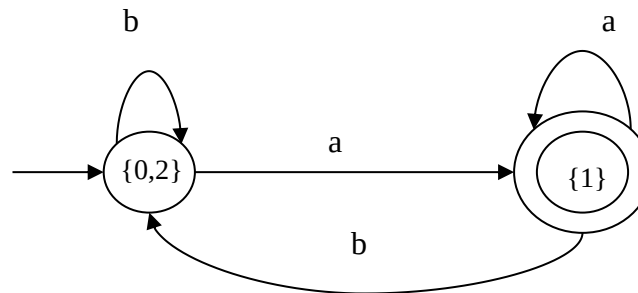
a	b
0->1	0->2
2->1	2->2

G_2 cannot be partitioned because

$$\text{move}(0, a) = 1 \quad \text{move}(0, b) = 2$$

$$\text{move}(2, a) = 1 \quad \text{move}(2, b) = 2$$

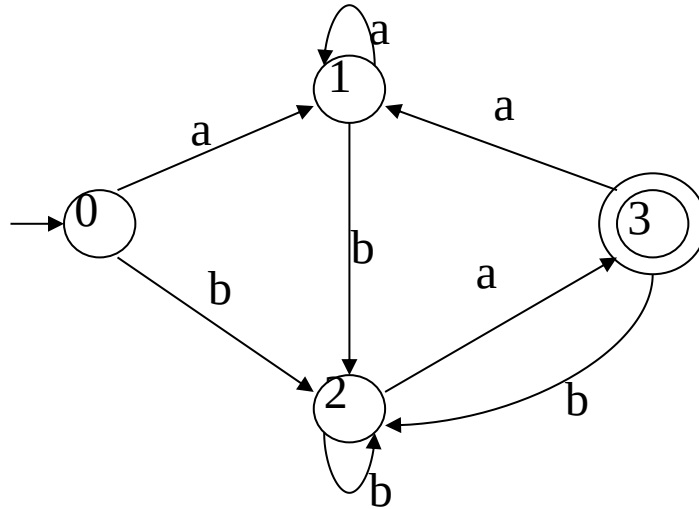
So, the minimized DFA (with minimum states)



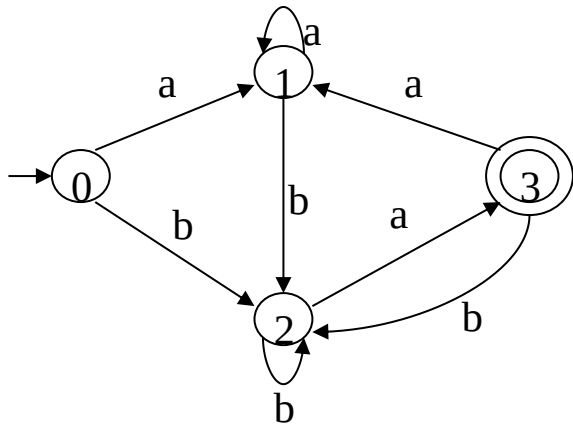
Minimizing DFA (Exercise)



- ◆ Minimizing DFA, and list state transition table



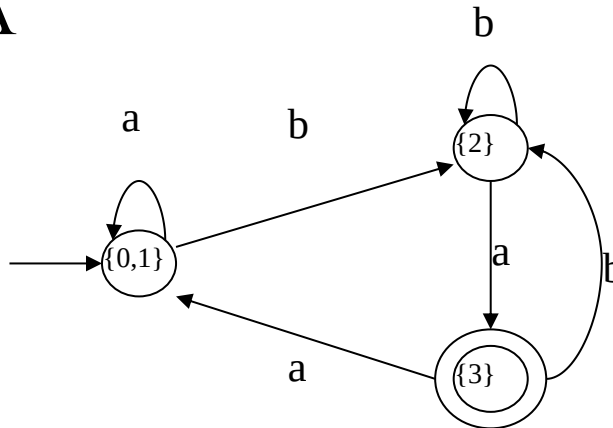
Minimizing DFA (exercise answer)



Groups: $\{0,1,2\}$ $\{3\}$
 $\{0,1\}$ $\{2\}$
no more partitioning

<u>a</u> <u>b</u>	
0->1	0->2
1->1	1->2
2->3	2->2

So, the minimized DFA



Exercises in class



◆ Construct DFA for the following regular expression in two ways:

$$1. a(a|b)^*(b| \in)$$

$$2. a(a|b)^*a(a|b)a$$

- The first way is to construct NFA by using Thompson's algorithm , then to construct DFA from NFA by using subset construction algorithm .
- The second way is to construct DFA from regular expression directly.
- Minimizing the states of DFA.